NLP Course
Search documentation Ctrl+K
EN 2,357

0. SETUP
1. TRANSFORMER MODELS
2. USING 🤗 TRANSFORMERS
Introduction
Behind the pipeline
Models
Tokenizers
Handling multiple sequences
Putting it all together
Basic usage completed!
End-of-chapter quiz
3. FINE-TUNING A PRETRAINED MODEL
4. SHARING MODELS AND TOKENIZERS
5. THE 🤗 DATASETS LIBRARY
6. THE 🤗 TOKENIZERS LIBRARY
7. MAIN NLP TASKS
8. HOW TO ASK FOR HELP
9. BUILDING AND SHARING DEMOS NEW
COURSE EVENTS

Join the Hugging Face community

and get access to the augmented documentation experience

Collaborate on models, datasets and Spaces

Faster examples with accelerated inference

Switch between documentation themes

Sign Up to get started

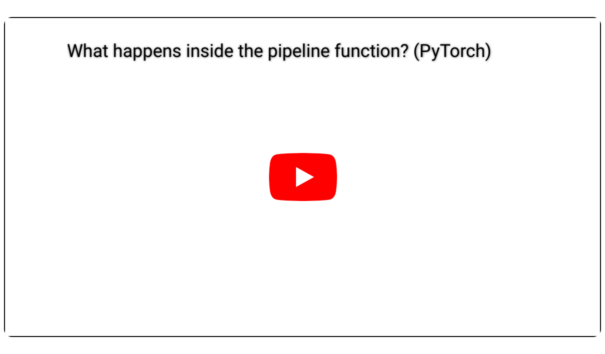
Pytorch TensorFlow

Ask a question Open in Colab Open In Studio Lab

Behind the pipeline

Behind the pipeline
Preprocessing with a tokenizer
Going through the model
A high-dimensional vector?
Model heads: Making sense out of numbers
Postprocessing the output

This is the first section where the content is slightly different depending on whether you use PyTorch or TensorFlow. Toggle the switch on top of the title to select the platform you prefer!



Let's start with a complete example, taking a look at what happened behind the scenes when we executed the following code in Chapter 1:

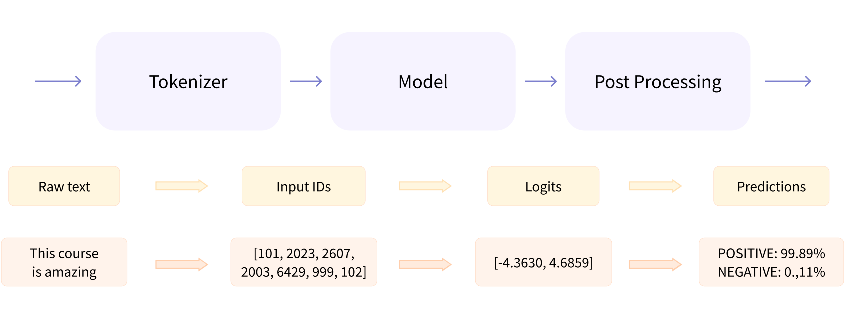
```
from transformers import pipeline

classifier = pipeline("sentiment-analysis")
classifier(
    [
        "I've been waiting for a HuggingFace course my whole life.",
        "I hate this so much!",
    ]
)
```

and obtained:

```
[{'label': 'POSITIVE', 'score': 0.9598047137260437},
 {'label': 'NEGATIVE', 'score': 0.9994558095932007}]
```

As we saw in Chapter 1, this pipeline groups together three steps: preprocessing, passing the inputs through the model, and postprocessing:



Let's quickly go over each of these.

Preprocessing with a tokenizer

Like other neural networks, Transformer models can't process raw text directly, so the first step of our pipeline is to convert the text inputs into numbers that the model can make sense of. To do this we use a *tokenizer*, which will be responsible for:

- Splitting the input into words, subwords, or symbols (like punctuation) that are called *tokens*
- Mapping each token to an integer
- Adding additional inputs that may be useful to the model

All this preprocessing needs to be done in exactly the same way as when the model was pretrained, so we first need to download that information from the Model Hub. To do this, we use the `AutoTokenizer` class and its `from_pretrained()` method. Using the checkpoint name of our model, it will automatically fetch the data associated with the model's tokenizer and cache it (so it's only downloaded the first time you run the code below).

Since the default checkpoint of the `sentiment-analysis` pipeline is `distilbert-base-uncased-finetuned-sst-2-english` (you can see its model card here), we run the following:

```
from transformers import AutoTokenizer

checkpoint = "distilbert-base-uncased-finetuned-sst-2-english"
tokenizer = AutoTokenizer.from_pretrained(checkpoint)
```

Once we have the tokenizer, we can directly pass our sentences to it and we'll get back a dictionary that's ready to feed to our model! The only thing left to do is to convert the list of input IDs to tensors.

You can use 🤗 Transformers without having to worry about which ML framework is used as a backend; it might be PyTorch or TensorFlow, or Flax for some models. However, Transformer models only accept *tensors* as input. If this is your first time hearing about tensors, you can think of them as NumPy arrays instead. A NumPy array can be a scalar (0D), a vector (1D), a matrix (2D), or have more dimensions. It's effectively a tensor; other ML frameworks' tensors behave similarly, and are usually as simple to instantiate as NumPy arrays.

To specify the type of tensors we want to get back (PyTorch, TensorFlow, or plain NumPy), we use the `return_tensors` argument:

```
raw_inputs = [
    "I've been waiting for a HuggingFace course my whole life.",
    "I hate this so much!",
]
inputs = tokenizer(raw_inputs, padding=True, truncation=True, return_tensors="pt")
print(inputs)
```

Don't worry about padding and truncation just yet; we'll explain those later. The main things to remember here are that you can pass one sentence or a list of sentences, as well as specifying the type of tensors you want to get back (if no type is passed, you will get a list of lists as a result).

Here's what the results look like as PyTorch tensors:

```
{
    'input_ids': tensor([
        [  101,  1045,  1005,  2310,  2042,  3403,  2005,  1037, 17662, 12172, 2607,  2026,  2878,  2166,
        [  101,  1045,  5223,  2023,  2061,  2172,   999,   102,     0,     0,     0,     0,     0,     0
    ]),
    'attention_mask': tensor([
        [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
        [1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0]
    ])
}
```

The output itself is a dictionary containing two keys, `input_ids` and `attention_mask`. `input_ids` contains two rows of integers (one for each sentence) that are the unique identifiers of the tokens in each sentence. We'll explain what the `attention_mask` is later in this chapter.

Going through the model

We can download our pretrained model the same way we did with our tokenizer. 🤗 Transformers provides an `AutoModel` class which also has a `from_pretrained()` method:

```
from transformers import AutoModel

checkpoint = "distilbert-base-uncased-finetuned-sst-2-english"
model = AutoModel.from_pretrained(checkpoint)
```

In this code snippet, we have downloaded the same checkpoint we used in our pipeline before (it should actually have been cached already) and instantiated a model with it.

This architecture contains only the base Transformer module: given some inputs, it outputs what we'll call *hidden states*, also known as *features*. For each model input, we'll retrieve a high-dimensional vector representing the **contextual understanding of that input by the Transformer model**.

If this doesn't make sense, don't worry about it. We'll explain it all later.

While these hidden states can be useful on their own, they're usually inputs to another part of the model, known as the *head*. In Chapter 1, the different tasks could have been performed with the same architecture, but each of these tasks will have a different head associated with it.

A high-dimensional vector?

The vector output by the Transformer module is usually large. It generally has three dimensions:

- **Batch size**: The number of sequences processed at a time (2 in our example).
- **Sequence length**: The length of the numerical representation of the sequence (16 in our example).
- **Hidden size**: The vector dimension of each model input.

It is said to be "high dimensional" because of the last value. The hidden size can be very large (768 is common for smaller models, and in larger models this can reach 3072 or more).

We can see this if we feed the inputs we preprocessed to our model:

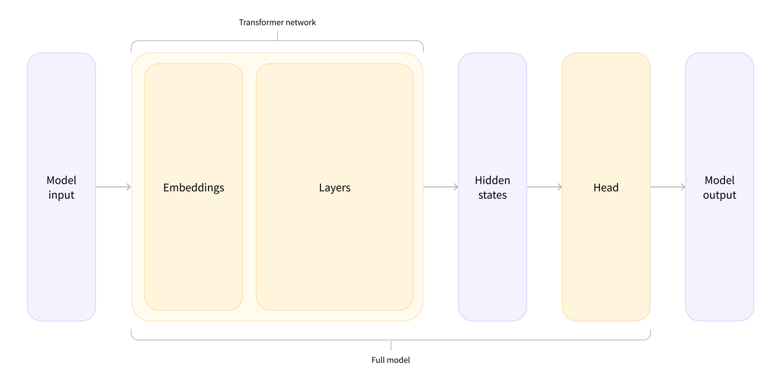
```
outputs = model(**inputs)
print(outputs.last_hidden_state.shape)
```

```
torch.Size([2, 16, 768])
```

Note that the outputs of 🤗 Transformers models behave like `namedtuple`s or dictionaries. You can access the elements by attributes (like we did) or by key (`outputs["last_hidden_state"]`), or even by index if you know exactly where the thing you are looking for is (`outputs[0]`).

Model heads: Making sense out of numbers

The model heads take the high-dimensional vector of hidden states as input and project them onto a different dimension. They are usually composed of one or a few linear layers:



The output of the Transformer model is sent directly to the model head to be processed.

In this diagram, the model is represented by its embeddings layer and the subsequent layers. The embeddings layer converts each input ID in the tokenized input into a vector that represents the associated token. The subsequent layers manipulate those vectors using the attention mechanism to produce the final representation of the sentences.

There are many different architectures available in 🤗 Transformers, with each one designed around tackling a specific task. Here is a non-exhaustive list:

- `*Model` (retrieve the hidden states)
- `*ForCausalLM`
- `*ForMaskedLM`
- `*ForMultipleChoice`
- `*ForQuestionAnswering`
- `*ForSequenceClassification`
- `*ForTokenClassification`
- and others 🤗

For our example, we will need a model with a sequence classification head (to be able to classify the sentences as positive or negative). So, we won't actually use the `AutoModel` class, but `AutoModelForSequenceClassification`:

```
from transformers import AutoModelForSequenceClassification

checkpoint = "distilbert-base-uncased-finetuned-sst-2-english"
model = AutoModelForSequenceClassification.from_pretrained(checkpoint)
outputs = model(**inputs)
```

Now if we look at the shape of our outputs, the dimensionality will be much lower: the model head takes as input the high-dimensional vectors we saw before, and outputs vectors containing two values (one per label):

```
print(outputs.logits.shape)
```

```
torch.Size([2, 2])
```

Since we have just two sentences and two labels, the result we get from our model is of shape 2 x 2.

Postprocessing the output

The values we get as output from our model don't necessarily make sense by themselves. Let's take a look:

```
print(outputs.logits)
```

```
tensor([[-1.5607,  1.6123],
        [ 4.1692, -3.3464]], grad_fn=<AddmmBackward>)
```

Our model predicted `[-1.5607, 1.6123]` for the first sentence and `[ 4.1692, -3.3464]` for the second one. Those are not probabilities but *logits*, the raw, unnormalized scores outputted by the last layer of the model. To be converted to probabilities, they need to go through a SoftMax layer (all 🤗 Transformers models output the logits, as the loss function for training will generally fuse the last activation function, such as SoftMax, with the actual loss function, such as cross entropy):

```
import torch

predictions = torch.nn.functional.softmax(outputs.logits, dim=-1)
print(predictions)
```

```
tensor([[4.0195e-02, 9.5980e-01],
        [9.9946e-01, 5.4418e-04]], grad_fn=<SoftmaxBackward>)
```

Now we can see that the model predicted `[0.0402, 0.9598]` for the first sentence and `[0.9995, 0.0005]` for the second one. These are recognizable probability scores.

To get the labels corresponding to each position, we can inspect the `id2label` attribute of the model config (more on this in the next section):

```
model.config.id2label
```

```
{0: 'NEGATIVE', 1: 'POSITIVE'}
```

Now we can conclude that the model predicted the following:

- First sentence: NEGATIVE: 0.0402, POSITIVE: 0.9598
- Second sentence: NEGATIVE: 0.9995, POSITIVE: 0.0005

We have successfully reproduced the three steps of the pipeline: preprocessing with tokenizers, passing the inputs through the model, and postprocessing! Now let's take some time to dive deeper into each of those steps.

✏️ **Try it out!** Choose two (or more) texts of your own and run them through the `sentiment-analysis` pipeline. Then replicate the steps you saw here yourself and check that you obtain the same results!